Convolutional Neural Networks

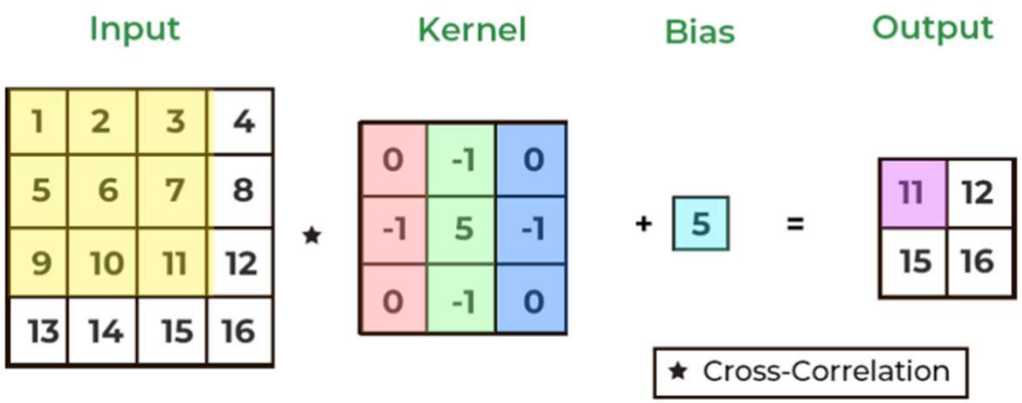
for Image Classification

Spring 2025-26

Outline

- 2D Convolution
- Pooling
- Convolutional Neural Networks for Image Classification
- The problems with FCNN for images
- CNN: Key Advantages
- Output Volume of Layers
- Hierarchical Abstractions
- Receptive Field of CNNs
- Dropout
- Batch Normalization

2D Convolution



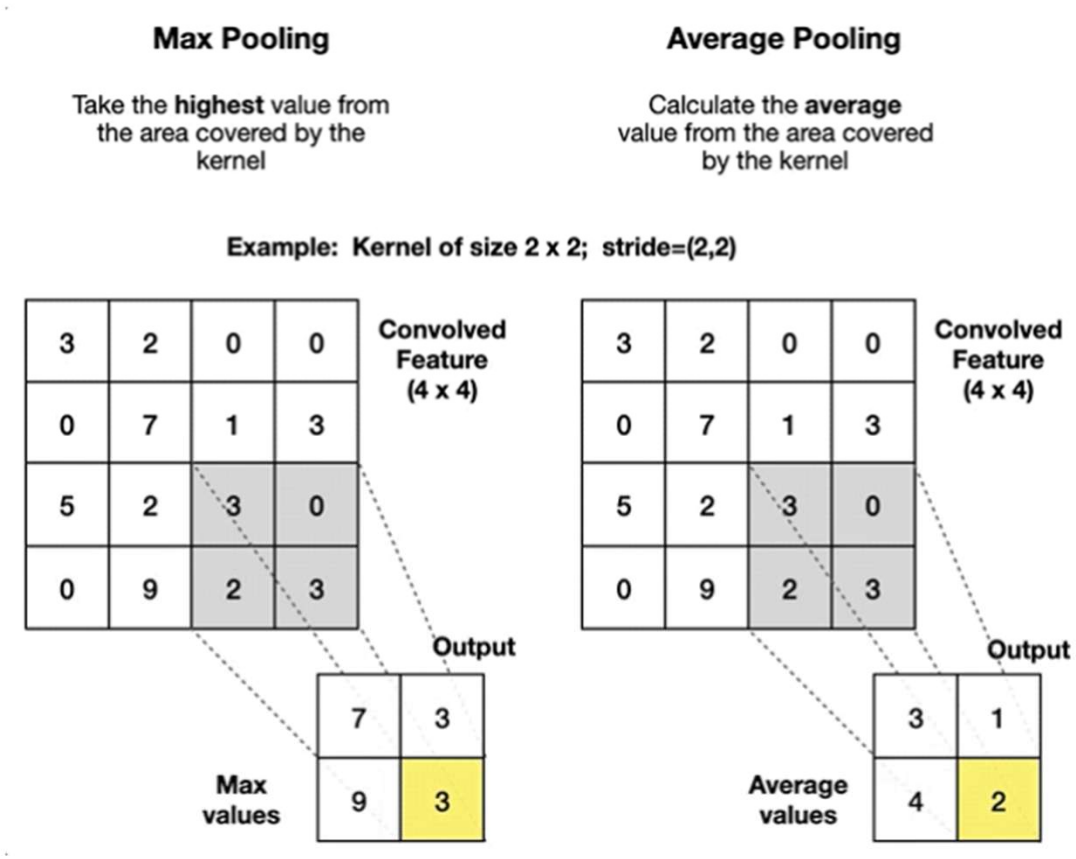
1 X 0 + 5 X -1 + 9 X 0 + 2 X -1 + 6 X 5 + 10 X -1 + 3 X 0 + 7 X -1 + 11 X 0 + 5 = 11

2 X 0 + 6 X -1 + 10 X 0 + 3 X -1 + 7 X 5 + 11 X -1 + 4 X 0 + 8 X-1 + 12 X 0 + 5 = 12

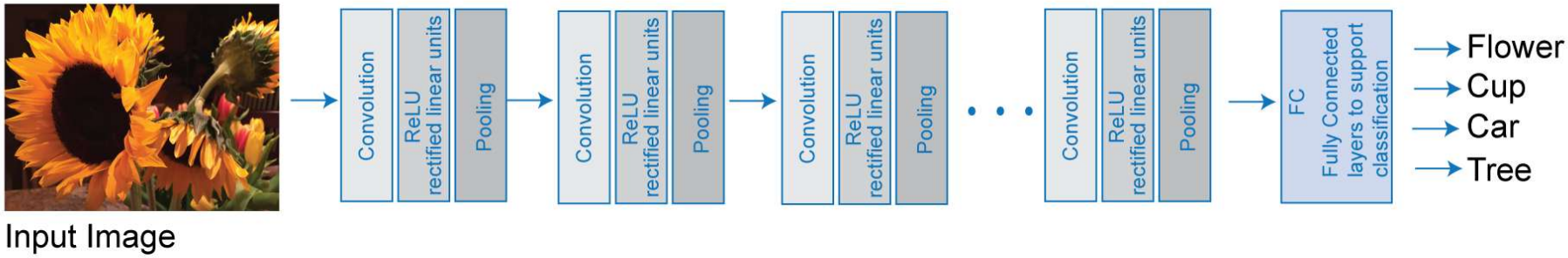
5 X 0 + 9 X -1 + 13 X 0 + 6 X -1 + 10 X 5 + 14 X -1 + 7 X 0 + 11 X -1 + 15 X 0 + 5 = 15

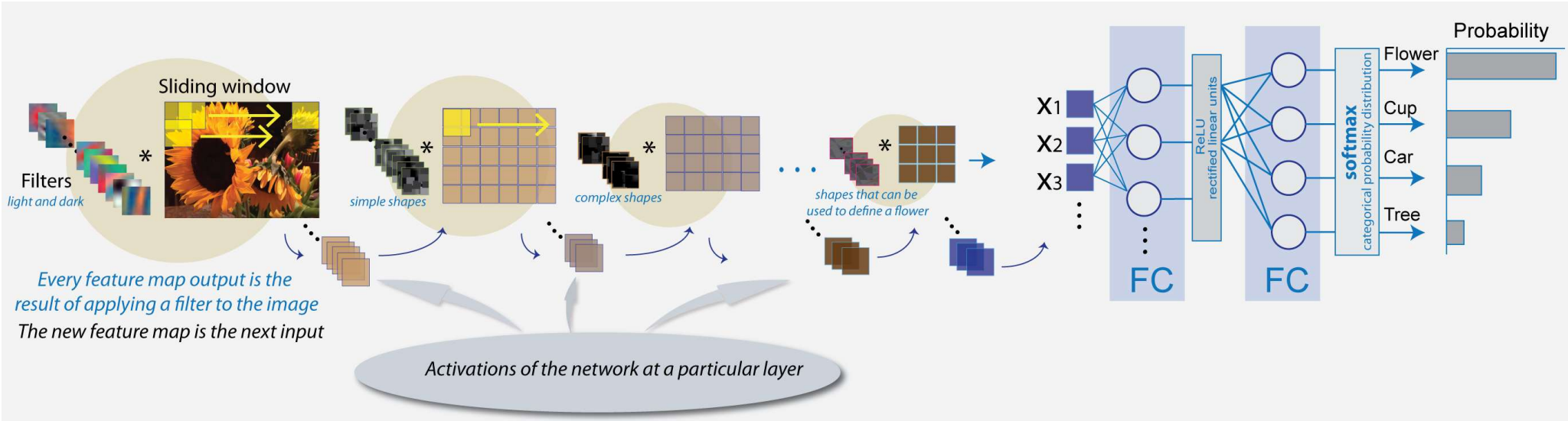
6 X 0 + 10 X -1 + 14 X 0 + 7 X -1 + 11 X 5 + 15 X -1 + 8X0 + 12X-1 + 16X0 + 5 = 16

Pooling


Max Pooling
Take the highest value from the area covered by the kernel
Average Pooling
Calculate the average value from the area covered by the kernel
Example: Kernel of size 2 x 2; stride=(2,2)
3 2 0 0
0 7 1 3
5 2 3 0
0 9 2 3
Convolved Feature (4 x 4)
Output
7 3
9 3
Max values
3 2 0 0
0 7 1 3
5 2 3 0
0 9 2 3
Convolved Feature (4 x 4)
Output
3 1
4 2
Average values

Convolutional Neural Networks





The Problem with Fully Connected Neural Network for Images

Parameter explosion: A 28*28 image fed to a

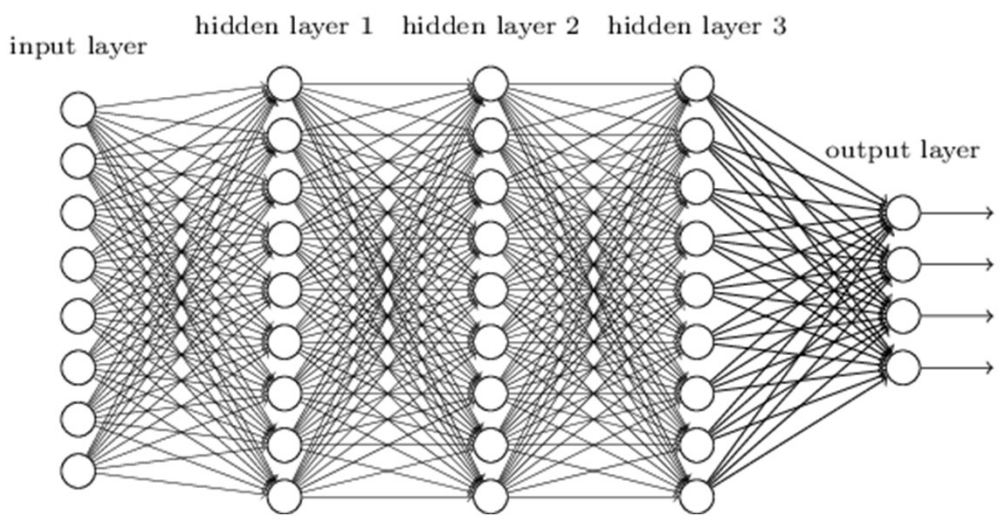
256 hidden layer needs

28 * 28 * 256 =

~0.2 million params in one layer alone.

No spatial awareness: Pixel (0,0) and pixel (28,28) are treated as equally related - spatial structure is destroyed.

Not translation invariant: An object shifted 2 pixels is an entirely different input vector.



CNN: Key Advantages

Parameter Sharing

The same small filter is applied at every spatial position. A 3×3 filter has only 9 weights regardless of image size.

Sparse Connectivity

Each output unit is connected to only a small receptive field (k×k patch), not all pixels.

Translational Invariance

Pooling after convolution makes detection robust to small shifts in the input.

Spatial Awareness

Feature maps maintain spatial layout; nearby features remain nearby.

0	0	0	0	0	0
0	2	3	1	4	0
0	3	1	3	2	0
0	3	0	1	3	0
0	0	2	0	1	0
0	0	0	0	0	0

*

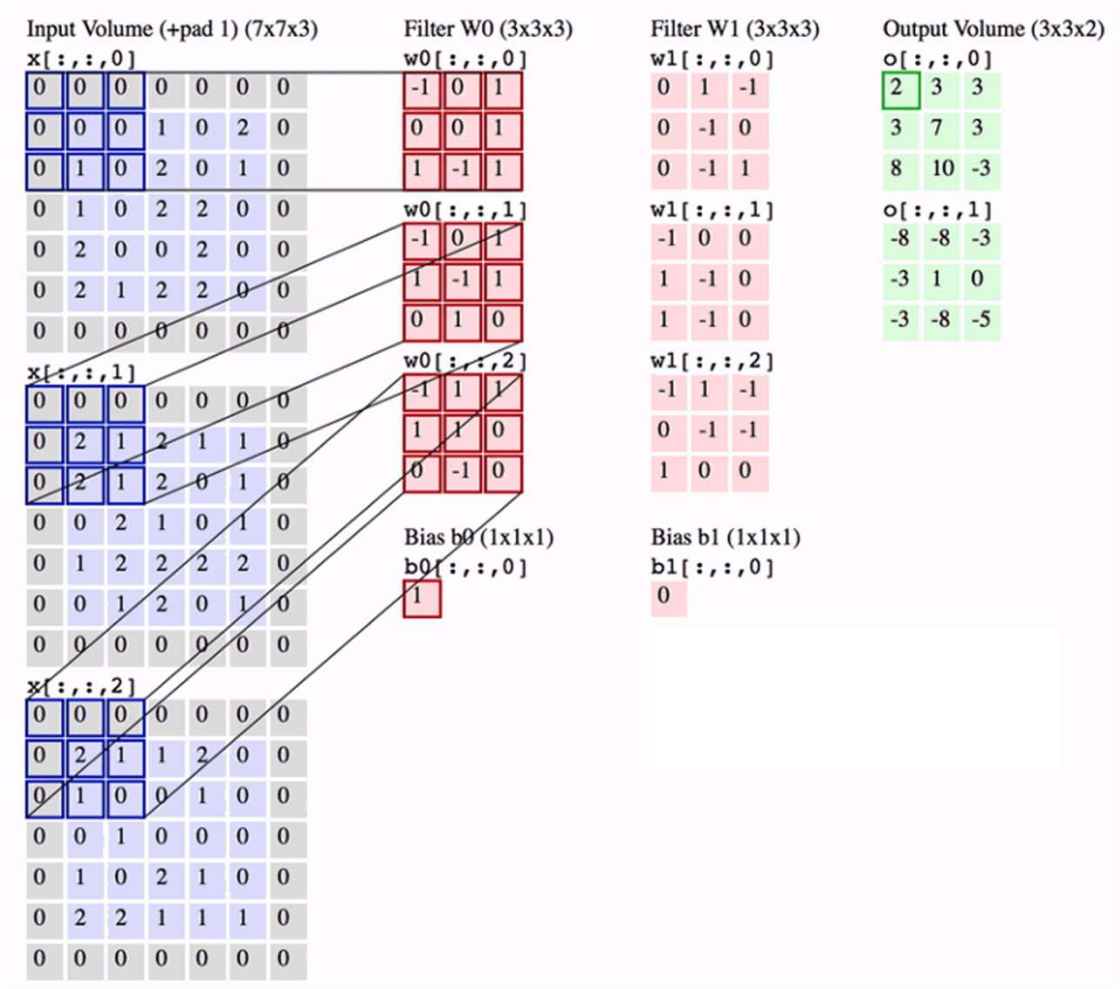
kernel

1	0	-1
1	0	-1
1	0	-1

=

-4	1	-2	4
-4	3	-5	5
-3	2	-3	4
-2	2	-2	1

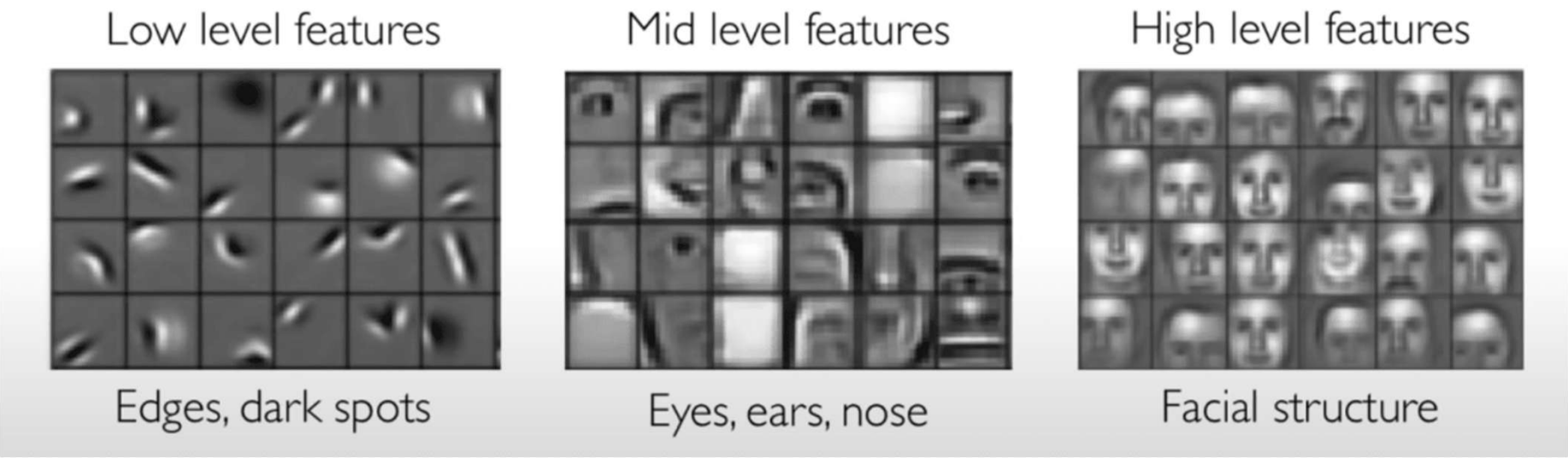
Output Volume of a Layer



CONV 1
Input Size (W_1 x H_1 x D_1) = 28 x 28 x 1
• Requires four hyperparameter: ○ Number of kernels, k = 16 ○ Spatial extend of each one, F = 5 ○ Stride Size, S = 1 ○ Amount of zero padding, P = 2
• Outputting volume of W_2 x H_2 x D_2 ○ W_2 = (28 – 5 + 2 (2)) / 1 + 1 = 28 ○ H_2 = (28 – 5 + 2 (2)) / 1 + 1 = 28 ○ D_2 = k
Output of Conv 1 (W_2 x H_2 x D_2) = 28 x 28 x 16

POOL 1
Input Size (W_2 x H_2 x D_2) = 28 x 28 x 16
• Requires two hyperparameter: ○ Spatial extend of each one, F = 2 ○ Stride Size, S = 2
• Outputting volume of W_3 x H_3 x D_2 ○ W_3 = (28 – 2) / 2 + 1 = 14 ○ H_3 = (28 – 2) / 2 + 1 = 14
Output of Pool 1 (W_3 x H_3 x D_2) = 14 x 14 x 16

Learning Hierarchical Abstractions



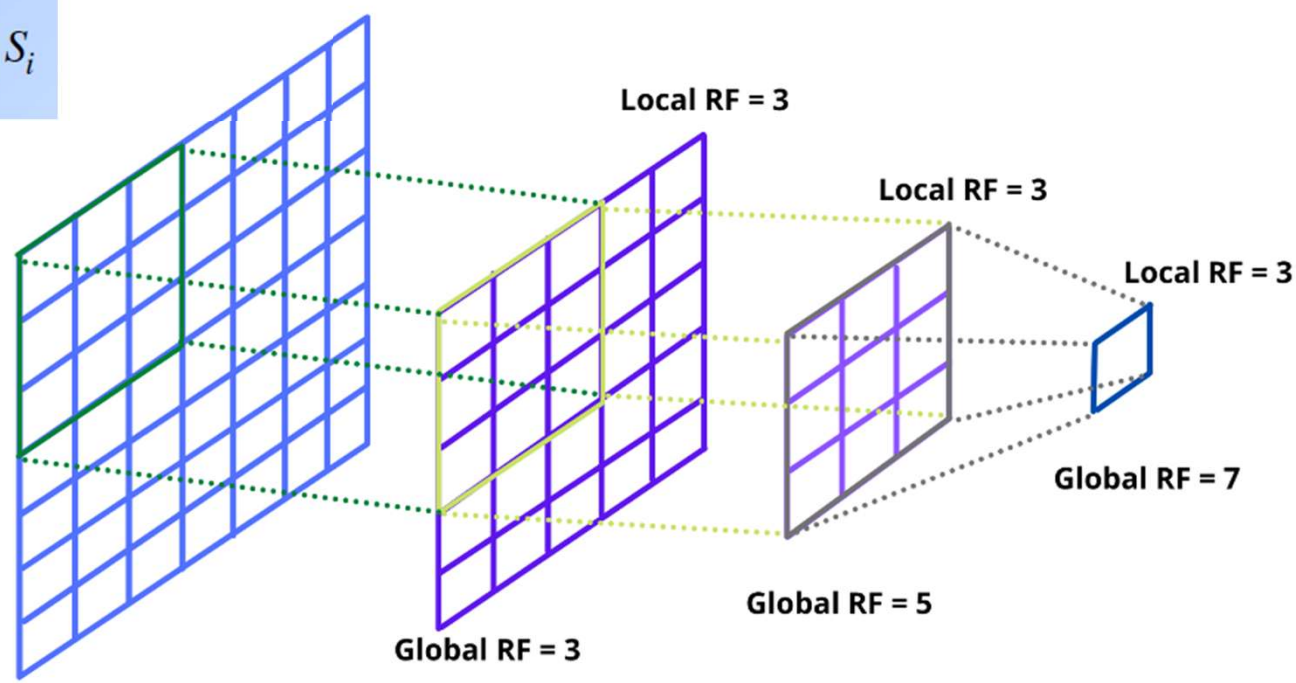
Receptive Field of CNNs

the area of the input image that is “visible” to this layer.

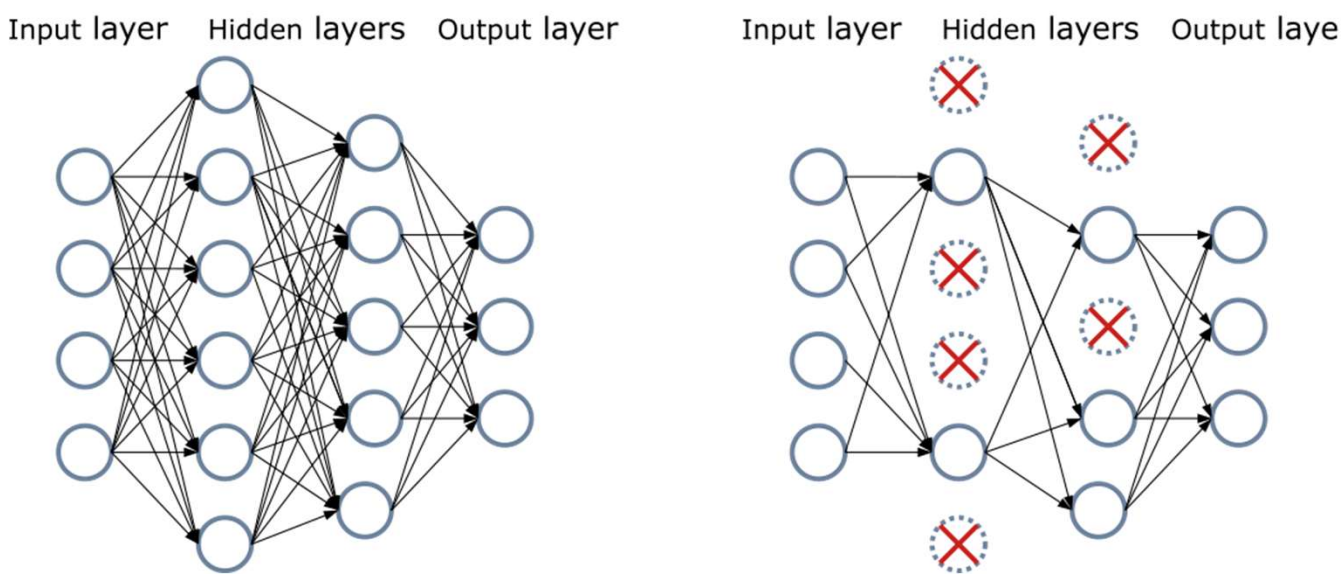
The receptive field size for each successive layer has the following recursive formula:

For first layer: $R_1 = K_1$, where K_1 is the size of the filter

For layer l: $R_l = R_{l-1} + (K_l - 1)\prod_{i=1}^{l-1} S_i$



Dropout



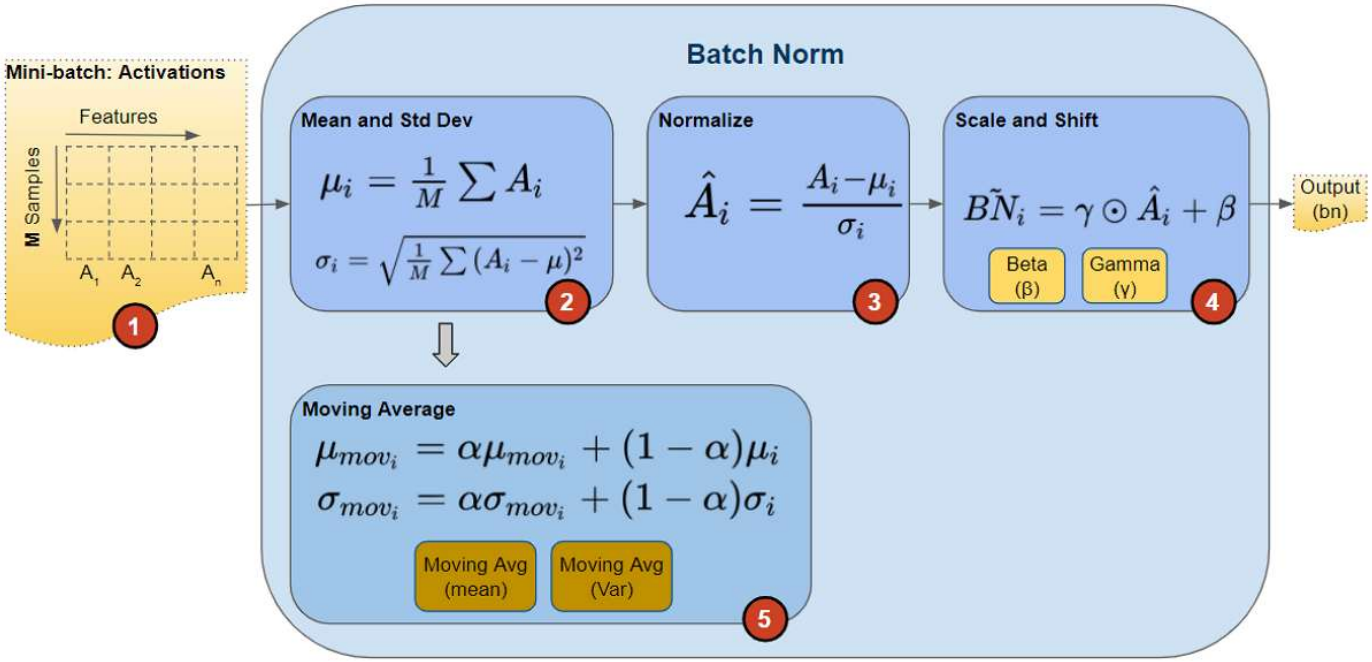
$$\tilde{x} = x \times m$$

- x: the input activation
- $m \sim \text{Bernoulli}(1 - \text{p})$ is a random binary mask
- $\tilde{x}$: output after applying dropout

The surviving activations are scaled up by $1/(1-p)$ during training, so no adjustment is needed at inference.

Batch Normalization

During training:



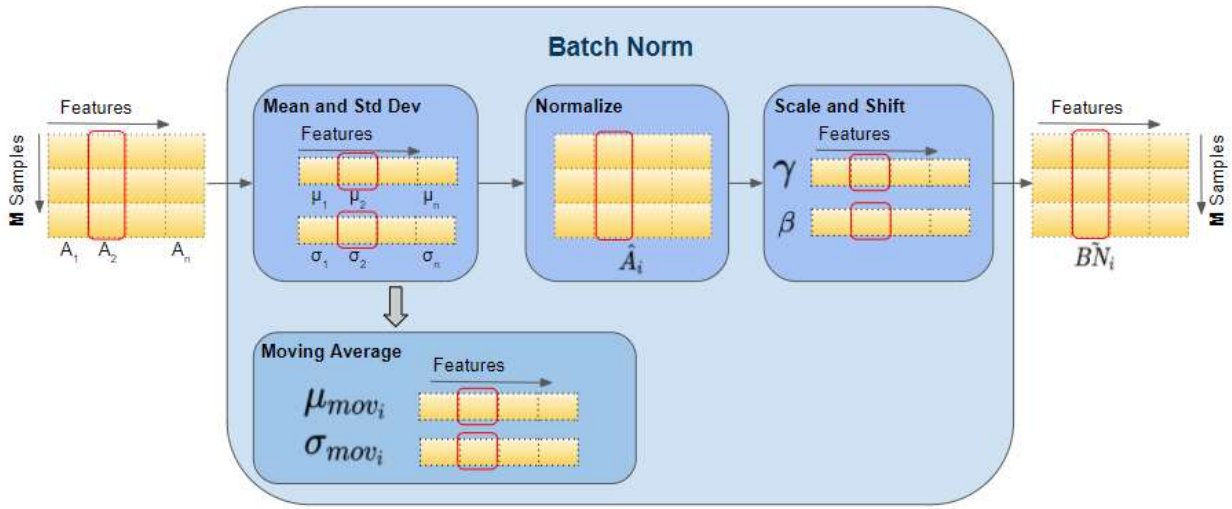
μ_i: *mean of mini − batch*, A_i: *features*, σ_i: *standard deviation of mini − batch*

$\mu_{mov\ i}$: *running mean*, σ_i: *running standard deviation*, $\propto$: *momentum parameter*

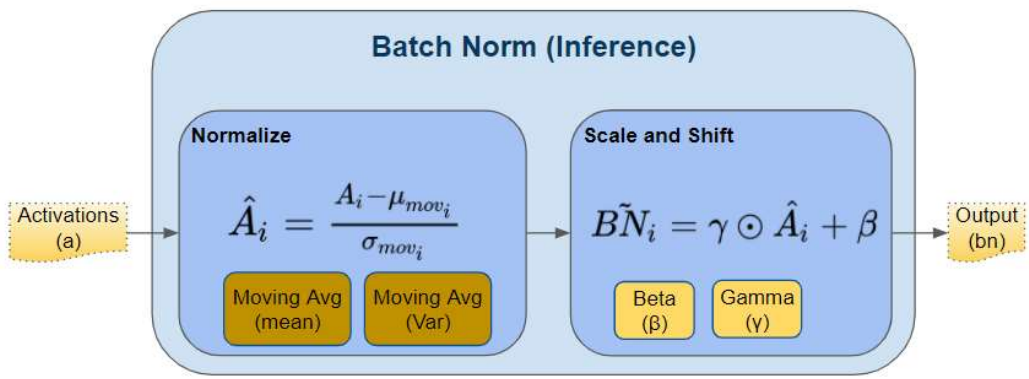
β, γ: *learnable parameters*

Batch Normalization

Vector shapes



During inference:



Thank You